

Code Review: Polity IPC Library v0.1

2026-05-02

Files Reviewed

- Polity/polity.py
 - Polity/message.py
 - Polity/record.py
 - Polity/monitor.py
 - Polity/jp-assistant-server.py
 - Polity/notes.txt
-

The Good

Record/Message is a genuinely clever idiom. A dict you can access with `.attribute` syntax, with strict schema enforcement (no new keys, type matching) — that makes message fields self-documenting and catches assignment errors at the Python level rather than at the wire level.

`monitor.py` showing just five lines to join a polity is exactly the right goal — the abstraction earns its weight.

The `background()` helper, `join_or_start_polity()` CQ/IAM handshake, and the `pause_then_announce()` anti-thundering-herd delay are all solid thinking.

Real Bugs

These would cause crashes or silent data loss.

1. Dead code in `_send()` — large messages silently fail

`polity.py` lines 116–119

```
except Exception as e:
    print(f"In polity._send: {e}")
    print('bad pack/unpack')
    return None                                # exits here
if len(packed_msg) > MAX_MESSAGE_LENGTH:      # UNREACHABLE
    self.send_long_message(msg)
```

The length check was meant to be in the `else:` block. As written, messages over 65 KB are silently dropped or truncated at the socket.

Fix: move the length check into the `else:` branch, before `sendto()`.

2. `listener_callback` is commented out but `__init__` wires it up

`polity.py` lines 74–75

```
self.listener = self.listener_callback # this method doesn't exist
```

Any code that passes `callback=` to `Polity()` will crash when the background thread starts.

Fix: implement the method, or raise `NotImplementedError` as a placeholder.

3. `leave()` passes a string to `_send()` instead of a dict

`polity.py` line 279

```
self._send('BYE') # _send() does msg['my_number'] = ... + TypeError
```

Fix: build a proper message dict (`type='BYE'` or `'SYS'`) before calling `_send()`.

4. `get_replies()` ignores its `timeout` parameter

`polity.py` lines 105–106

```
def get_replies(self, timeout=0)->Message:
    return self.replies.get() # blocks forever; timeout unused
```

Fix: `self.replies.get(timeout=timeout)` and handle `queue.Empty`.

5. `PORT_NUMBER` and `self.port` are never used

`polity.py` lines 29, 94

```
PORT_NUMBER = 4242 # declared but ignored
server_address = ('', 10000) # hardcoded, ignores self.port
```

The `port` argument to `__init__` is stored as `self.port` but the socket always binds to 10000. You cannot run instances on different ports.

Fix: use `self.port` in the `server_address` tuple.

Design Issues

6. `reply()` mutates the caller's message in-place

polity.py line 169

```
input_msg.update(m)      # overwrites the received message object
self._send(input_msg)
```

The caller's object is silently modified. Should construct a new `Message` for the reply rather than overwriting the incoming one.

7. `send_long_message()` raises `ZeroDivisionError` as a placeholder

polity.py line 132

```
return 1/0      # produces a confusing traceback
```

Fix: `raise NotImplementedError("send_long_message not implemented")`

8. `pack()` silently strips all falsy values

polity.py line 222

```
msg = {key:value for (key, value) in msg.items() if value}
```

This strips `my_number=0`, `reply_to=0`, `body_checksum=0`, `continued=False`, `packets=0`, and any empty-string field like `to=' '`. Most are harmless because they match the `Message` template defaults, but a `sequence_number` of 0 on the very first message is lost, and it's invisible behavior.

9. Duplicate self-message filtering

`message_handler()` (line 287) and `doesn't_concern_us()` (line 247) both filter `msg['from'] == self.id`. The `# FIXME` comment in `listener_queue()` saying it “bypasses `message_handler`” is stale — `_receive()` calls `message_handler()` every time.

Schema Observation (from `notes.txt` / captured message)

`from_ip` appears in every sent message (added by `message()`) but is absent from the `Message` template in `message.py`. This works in practice because CPython's `dict.update()` uses a fast path when given a plain dict argument that bypasses `Record.__setitem__`'s “no new keys” check.

The implication: **Record's schema enforcement only applies to direct attribute/item assignment** (`msg.x = y` or `msg['x'] = y`), not to initialization via `update()`. `Record` is a soft schema, not a hard one.

Fix: add 'from_ip': '' to the msg template dict in `message.py` so the wire format and the schema agree.

Notes.txt Items

my_number naming: the notes are right — `sequence_number` is clearer. If you want to distinguish directions, `seq` / `reply_to_seq` pair cleanly.

SPI / SPO message types for speech I/O are a natural fit for Persona. The pipeline stages map directly to roles/types:

Type	Meaning
SPI	Speech in — STT output, goes to dispatcher
SPO	Speech out — text going to TTS

These should be added to `Polity.msg_types` and `Polity.roles`.

On Using Polity for Persona IPC

The design fits well. The pipeline (STT → dispatcher → LLM → TTS) maps naturally to named roles, and the CQ/IAM discovery means processes can start in any order without a coordinator. UDP multicast on loopback adds negligible overhead for localhost use and keeps the option to span machines later.

Priority fixes before wiring Persona to it:

1. Bug #5 — port always 10000 (needed if running multiple polities)
2. Bug #1 — silent large-message loss (LLM responses can be long)
3. Bug #2 — callback crash (if you plan to use the callback path)
4. Schema fix — add `from_ip` to `Message` template

The rest can wait until the pipeline is running.